



Polimorfismo

Tratti e programmazione generica

Lezioni 11 12
Corso 1



Polimorfismo

- **Polimorfismo**, deriva dal greco πολύς (molto) e μορφή (forma), dunque significa **multiforme**.
- In informatica:
 - Associare comportamenti comuni a oggetti i cui tipi sono molto diversi tra di loro.



Polimorfismo

- Sebbene il processo di analisi di un dominio applicativo tenda a favorire la sua suddivisione in una molteplicità di tipi distinti (classi, strutture, funzioni, ecc.), la necessità di minimizzare il codice scritto spinge verso l'identificazione di pattern comuni
 - Che possano essere condivisi tra tali tipi per unificare la struttura del codice
 - Principio DRY - *Don't Repeat Yourself* (parafrasando: cercare di scrivere le cose una volta sola)
- La soluzione individuata è il **polimorfismo**: capacità offerta dai linguaggi di **associare comportamenti comuni ad un insieme di tipi differenti**
 - Può basarsi sul paradigma della **programmazione generica**, dove è possibile formulare funzioni o strutture dati in cui uno o più tipi non sono specificati per nome, ma tramite simboli astratti, abilitando così la scrittura di codice in grado di operare con una molteplicità di tipi
 - Oppure può sfruttare la definizione di **interfacce** comuni, che possono essere implementate dai singoli tipi: questo consente di fare riferimento all'interfaccia invece che al tipo concreto.

Tratto

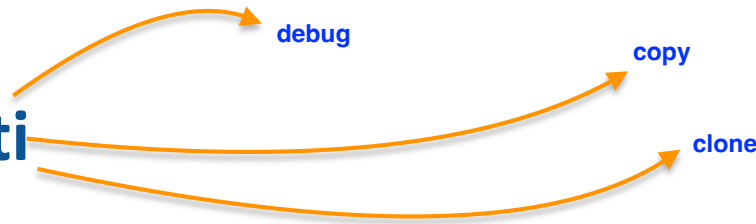
- Collezione di **metodi** che definiscono un comportamento che un **tipo** può **implementare**
- Il tratto esprime la capacità che un tipo offre di avere un certo comportamento.

tratto: elenco di metodi che descrivono i possibili comportamenti di un oggetto tramite parametri e valori di ritorno senza necessariamente implementare tutti i dettagli.

Tramite polimorfismo un tratto posso implementarlo su un tipo e in questo caso devo dettagliare i metodi che caratterizzano il tratto.

Il compilatore, analizzando un tratto applicato ad un certo tipo, va a vedere il metodo associato

Tratti



posso scrivere tratti ad hoc e
utilizzarli per i tipi che sto usando

implementare i tratti non
influisce sull'occupazione
della memoria, il costo è 0

- I tratti in Rust costituiscono l'equivalente delle interfacce di Java e C# o delle classi astratte pure in C++
 - Un tratto definisce un insieme di metodi, eventualmente associando loro un'implementazione di default
- Un tratto esprime la capacità di un tipo di eseguire una certa funzionalità
 - Un tipo che implementa `std::io::Write` può scrivere dei byte su file
 - Un tipo che implementa `std::iter::Iterator` può produrre una sequenza di valori
 - Un tipo che implementa `std::clone::Clone` può creare copie del proprio valore
 - Un tipo che implementa `std::fmt::Debug` può essere stampato tramite `println!()` usando il formato `{:?}` e trasforma il tipo in una stringa visualizzabile
- A differenza di quanto accade in C++ o Java, se si invoca su un valore una funzione relativa ad un tratto, non si ha - normalmente - un costo aggiuntivo (**zero-cost abstraction**)
 - Né gli oggetti che implementano tratti hanno una penalità in termini di memoria per ospitare il puntatore alla VTABLE
 - Tale costo si presenta solo quando si crea esplicitamente un riferimento dinamico (`&dyn TraitName`) quando chiedo al compilatore di dimenticarsi del tipo, ma di considerare solo in virtù del tratto che implementa.

Definire ed usare un tratto

qui metto i metodi che descrivono il tratto e qui intendiamo solo ed esclusivamente delle SIGNATURE, delle firme

per usare un tratto su un tipo specifico devo implementare il tratto per quel tipo

- Si definisce un tratto con la sintassi
 - `trait SomeTrait { fn someOperation(&mut self) -> SomeResult; ... }`
- Una struttura dati concreta, come struct od enum, può esplicitamente dichiarare di implementare un dato tratto attraverso il blocco seguente
 - `impl SomeTrait for SomeType { ... }` → questo significa letteralmente implemento il tratto per un tipo
- **Dato un valore il cui tipo implementa un tratto**, è possibile **invocare su tale valore i metodi del tratto, con la normale** sintassi basata sul `'.'`
 - A condizione che il tratto sia stato dichiarato nello stesso crate o che sia stata importato attraverso il costrutto
 - `use SomeNamespace::SomeTrait;`
 nelle graffe specifico i metodi per il tratto
 - Alcuni tratti (come Clone e Iter) non necessitano di essere importati esplicitamente in quanto fanno parte di una porzione di codice della libreria standard (`std::prelude`, il cosiddetto **preludio**) che viene importato automaticamente in ogni crate
 - L'implementazione del tratto può essere fatta o dove viene definito il tratto o dove viene definita la struttura dati.

Esempio

```
trait Summarizable {  
    fn summary(&self) -> String;  
}  
  
impl Summarizable for f64 {  
    fn summary(&self) -> String {  
        format!("{:.4}", self)  
    }  
}  
  
impl Summarizable for &str{  
    fn summary(&self) -> String {  
        if self.len() > 5 {  
            format!("{}", &self[..=5])  
        }  
        else {  
            self.to_string()  
        }  
    }  
}  
  
fn main() {  
    let n = 1.0/3.0;  
    println!("{}", n.summary());  
    println!("{}", "Hello, World".summary());  
}
```

nell'implementazione specifica dei metodi devo riprendere le signature nella definizione del tratto

trait0.rs

format è una macro che produce uno String. Quindi la chiamata di summary è coerente perchè converte il riferimento

qui utilizzo il metodo summary per il tipo f64

crea una slice con i primi 5 caratteri

qui utilizzo il metodo summary per il tipo &str

Definire ed usare un tratto

- Un metodo è una funzione che utilizza come primo parametro la parola chiave **self** o una sua variazione (**&self**, **&mut self**, ...)
 - Il tipo del parametro self può anche essere **Box<Self>**, **Rc<Self>**, **Arc<Self>**, **Pin<Self>**
- I metodi sono invocati con l'operatore **.** (punto) sul ricevitore su cui è applicato il metodo

```
trait T2 {  
    fn takes_self(self);  
    fn takes_immut_self(&self);  
    fn takes_mut_self(&mut self);  
}
```

equivalenti

```
trait T2 {  
    fn takes_self(self: Self);  
    fn takes_immut_self(self: &Self);  
    fn takes_mut_self(self: &mut Self);  
}
```

self può essere o nello stack o nello heap, e in questo caso uso il suo puntatore che è il Box. Tuttavia il Self deve esserci, e il Box si usa perchè tutto ciò che gli sta dentro vive nello heap. Self (con S maiuscolo) è il tipo. Ma il Self può essere un Box di Self, e quindi sta nello Heap. Peranto, per accedere a Self mi serve utilizzare il puntatore

il metodo è applicato al tipo. Il tipo può essere o nello stack oppure può essere stato incapsulato nello Heap e quindi per accederci serve il puntatore


```
trait HasArea {  
    fn get_area(&self) -> f64;  
}
```

```
struct Circle { radius: f64}  
impl HasArea for Circle {  
    fn get_area(&self) -> f64 {  
        std::f64::consts::PI * (self.radius * self.radius)  
    }  
}
```

```
struct Rectangle {    width: f64,    height: f64,}  
impl HasArea for Rectangle {  
    fn get_area(&self) -> f64  
    {  
        self.width * self.height  
    }  
}  
  
fn main() {  
    let circle = Circle { radius: 3.0 };  
    let rectangle = Rectangle { width: 4.0, height: 5.0 };  
    println!("L'area del cerchio è: {}", circle.get_area());  
    println!("L'area del rettangolo è: {}", rectangle.get_area());  
}
```



spazio in cui stanno tutte le costanti

```
trait Scalabile {  
    fn scala(&self, num: f32) -> Self ;  
}
```

```
struct Punto {  
    x: f32,  
    y: f32,  
}
```

```
impl Scalabile for Punto {  
    fn scala(&self, num:f32) -> Self {  
        Self {  
            x: self.x*num,  
            y: self.y*num,  
        }  
    }  
}
```

```
fn main() {  
    let punto_originale = Punto { x: 10.0, y: 20.0 };  
    let punto_scalato = punto_originale.scala(0.5);  
  
    println!("Punto originale - x: {}, y: {}", punto_originale.x, punto_originale.y);  
    println!("Punto scalato - x: {}, y: {}", punto_scalato.x, punto_scalato.y);  
}
```



La parola chiave **Self**, nella definizione di un tratto, si riferisce al tipo che lo implementerà
Permette di creare una istanza del tipo che implementa il tratto

Definire ed usare un tratto: senza self

- Se una delle funzione definite da un tratto non usa, come primo parametro, né **self** né un suo derivato (**&self**, **&mut self**, ...), questa non è legata all'istanza del tipo che la implementa e può essere invocata usando come prefisso il nome del tratto o il nome del tipo che la implementa

produce il valore di default di quel tipo, per cui è di tipo Self. Serve per creare un istanza che ha un valore predefinito.

```
trait Default {  
    fn default() -> Self  
}
```

```
fn main() {  
    let zero: i32 = Default::default();  
    let zero_again = i32::default();  
}
```

nome del tratto

Text

in questo caso è necessario specificare il tipo qui, perchè il compilatore non lo può conoscere

```
trait Inizializzabile {  
    fn inizializza(a:i32, b:i32) -> Self;  
}
```

```
#[derive(Default)]  
struct Punto {
```

```
    x: i32,  
    y: i32,  
}
```

è necessario sempre specificare che una struct può usufruire del tratto Default

```
impl Inizializzabile for Punto {  
    fn inizializza(a:i32, b:i32) -> Self {  
        Punto { x: a, y: b }  
    }  
}
```

i tipi composti possono derivare i tratti standard se sono fatit in un certo modo. Poichè la struct Punto contiene solo tipi elementari, allora è in grado applicare il tratto default per ogni singolo campo

```
fn main() {  
    let punto = Punto::inizializza(5, 10);  
    let punto1:Punto = Default::default();  
    println!("Punto inizializzato: ({} , {})", punto.x, punto.y);  
    println!("Punto inizializzato con Default: ({} , {})", punto1.x, punto1.y);  
}
```



Tipo associato ad un tratto

- Un tratto può avere uno o più **tipi associati** (oltre al tipo Self specifico del tratto)
- Nell'**implementazione** specifica a quale tipo si riferisce (detto **tipo concreto**)
 - Questo permette alle funzioni del tratto di fare riferimento, in modo astratto, a tali tipi che dovranno essere poi specificati nel contesto del tipo che implementa il tratto stesso
- Firma generica comune, le singole implementazione forniscono i dettagli sul tipo.

```
trait T3 {  
    type AssociatedType;  
    fn f(arg: Self::AssociatedType);  
}
```

```
struct SomeType;  
impl T3 for SomeType {  
    type AssociatedType = i32;  
    fn f(arg: Self::AssociatedType) {}  
}
```

```
struct OtherType;  
impl T3 for OtherType {  
    type AssociatedType = &str;  
    fn f(arg: Self::AssociatedType) {}  
}
```

```
fn main() {  
    SomeType::f(1234);  
    OtherType::f("Hello, Rust!");  
}
```

```
trait Convertibile {
    type Output;
    fn converti(&self) -> Output;
}
```



output è un tipo. Quando implemento
il tratto dettaglio che tipo è. Quindi
è come se fosse un parametro

bisogna dettagliare che tipo è output

```
struct NumeroIntero { valore: i32}
impl Convertibile for NumeroIntero {
    type Output = f64;
    fn converti(&self) -> Output {
        self.valore as Output }
}
```

```
struct NumeroReale { valore: f64}
impl Convertibile for NumeroReale {
    type Output = i32;
    fn converti(&self) -> Output {
        self.valore as Output }
}
```

```
fn main() {
    let numero = NumeroIntero { valore: 42 };
    let valore_convertito = numero.converti();
    println!("Valore convertito: {}", valore_convertito);
```

```
    let numero = NumeroReale { valore: 42.3 };
    let valore_convertito = numero.converti();
    println!("Valore convertito: {}", valore_convertito);
}
```

```
error[E0425]: cannot find type `Output` in this scope
--> src\main.rs:9:27
```

```
9 |         fn converti(&self) -> Output;
    |                               ^^^^^^
```

help: you might have meant to use the associated type

```
9 |         fn converti(&self) -> Self::Output;
    |                               ++++++
```

help: consider importing this struct

```
7 + use std::process::Output;
    |
```

```

trait Convertibile {
    type Output;
    fn converti(&self) -> Self::Output;
}

struct NumeroIntero { valore: i32}
impl Convertibile for NumeroIntero {
    type Output = f64;
    fn converti(&self) -> Self::Output {
        self.valore as Self::Output }
}

struct NumeroReale { valore: f64}
impl Convertibile for NumeroReale {
    type Output = i32;
    fn converti(&self) -> Self::Output {
        self.valore as Self::Output }
}

fn main() {
    let numero = NumeroIntero { valore: 42 };
    let valore_convertito = numero.converti();
    println!("Valore convertito: {}", valore_convertito);

    let numero = NumeroReale { valore: 42.3 };
    let valore_convertito = numero.converti();
    println!("Valore convertito: {}", valore_convertito);
}

```



Self::TipoAssociato

Questa sintassi viene utilizzata per riferirsi al tipo associato specifico per l'implementazione corrente, ossia "il tipo Output che ho definito per me stesso (Self) in questa implementazione".

Implementazione di default

nella definizione del tratto posso anche mettere un metodo. L'implementazione del tratto può essere chiamata e quindi è una forma di override

- Nella definizione di un tratto è lecito indicare, per una data funzione, un'implementazione di default
 - Le funzioni che implementano il tratto saranno libere di adottarla o potranno sovrascriverla con altro codice (**override**), purché venga rispettata la firma delle funzione (tipo dei parametri e del valore di ritorno)
 - Questo è particolarmente comodo in quelle situazioni in cui un dato metodo può essere implementato in funzione di altri metodi del tratto

```
trait T4 {  
  fn f() { println!("default"); }  
}
```

```
struct SomeType;  
impl T4 for SomeType { }  
//uso dell'implementazione di default
```

```
struct OtherType;  
impl T4 for OtherType {  
  fn f() { println!("Other"); }  
}
```

```
fn main() {  
  SomeType::f(); // default  
  OtherType::f(); // Other  
}
```



```

trait Moltiplicabile {
    fn moltiplica(&self, factor: i32) -> i32;
    fn moltiplica_per_due(&self) -> i32 {
        self.moltiplica(2) }
}

struct Numero {valore: i32}
struct Numero2 { valore: i32, altro: i32 }
impl Moltiplicabile for Numero {
    fn moltiplica(&self, factor: i32) -> i32 {
        self.valore * factor
    }
    fn moltiplica_per_due(&self) -> i32 {
        println!("Moltiplicazione per 2");
        self.moltiplica(2) // Ridefinizione del metodo con un comportamento diverso
    }
}

impl Moltiplicabile for Numero2 {
    fn moltiplica(&self, factor: i32) -> i32 {
        self.valore * factor
    }
}

fn main() {
    let numero2 = Numero2 { valore: 5, altro: 1 };
    let numero = Numero { valore: 7};
    println!("Risultato con Tipo Numero: {}", numero.moltiplica_per_due());
    println!("Risultato con Tipo Numero2: {}", numero2.moltiplica_per_due());
}

```



Sotto-tratti e super-tratti

- La notazione `trait Subtrait: Supertrait {...}` indica che i tipi che implementano *Subtrait* **devono** implementare anche *Supertrait*
 - Le due implementazioni sono tra loro indipendenti
- Il sottotratto include tutte le funzionalità del supertratto e ne può aggiungere di proprie

```
trait Supertrait {  
    fn Sup(&self) {println!("In super");}  
    fn g(&self) {}  
}  
  
trait Subtrait: Supertrait {  
    fn Sub(&self) {println!("In sub");}  
    fn h(&self) {}  
}
```

```
struct SomeType;  
impl Supertrait for SomeType {}  
impl Subtrait for SomeType {}  
  
fn main() {  
    let s = SomeType;  
    s.Sup();  
    s.Sub();  
}
```

```
trait Animali { fn fanno_verso(&self); }
```

```
trait Mammiferi: Animali {  
    fn si_cibano(&self); }
```

```
trait Uccelli: Animali {  
    fn volano(&self);  
}
```

```
struct Dog { nome: String }  
struct Crow { weight: f32 }
```

```
impl Animali for Crow {  
    fn fanno_verso(&self) {println!("Cra Cra");}  
}
```

```
impl Animali for Dog {  
    fn fanno_verso(&self) {println!("Bau Bau"); }  
}
```

```
impl Mammiferi for Dog {  
    fn si_cibano(&self) {  
        println!("I cuccioli di cani sono allattati.");  
    }  
}
```

```
impl Uccelli for Crow {  
    fn volano(&self) { println!("I corvi volano."); }  
}
```

```
fn main() {  
    let dog = Dog {nome: "Fido".to_string() };  
    let corvo = Crow { weight: 2.20 };  
  
    dog.fanno_verso();  
    dog.si_cibano();  
    corvo.fanno_verso();  
    corvo.volano();  
}
```



codice senza alcun senso logico

```
trait Stampabile {
    fn stampa(&self);
}

trait Intestabile: Stampabile {
    fn stampa_intestazione(&self);
}

struct Documento {
    titolo: String,
    contenuto: String,
}

impl Stampabile for Documento {
    fn stampa(&self) {
        println!("Contenuto: {}", self.contenuto);
    }
}

impl Intestabile for Documento {
    fn stampa_intestazione(&self) {
        println!("--- {} ---", self.titolo);
        self.stampa(); // Possiamo chiamare i metodi di Stampabile
    }
}
```

```
fn main() {
    let mio_documento = Documento {
        titolo: String::from("Il Mio Rapporto"),
        contenuto: String::from("Contenuto bla bla"),
    };

    mio_documento.stampa();
    println!("---");
    mio_documento.stampa_intestazione();
}
```



Sotto-tratti e super-tratti (II)

```
trait Supertrait {  
    fn f(&self) {println!("In super");}  
    fn g(&self) {}  
}  
  
trait Subtrait: Supertrait {  
    fn f(&self) {println!("In sub");}  
    fn h(&self) {}  
}
```

```
struct SomeType;  
impl Supertrait for SomeType {}  
impl Subtrait for SomeType {}  
  
fn main() {  
    let s = SomeType;  
    s.f(); //Errore: chiamata ambigua  
    <SomeType as Supertrait>::f(&s);  
    <SomeType as Subtrait>::f(&s);  
}
```

```

trait Animali {
    fn fanno_verso(&self) {
        println!("Non fanno versi");
    }
}

trait Mammiferi: Animali {
    fn fanno_verso(&self);
    fn si_cibano(&self);
}

trait Uccelli: Animali {
    fn volano(&self); }

struct Dog { nome: String }
struct Crow { weight: f32 }

impl Animali for Crow { }
impl Animali for Dog { }

impl Mammiferi for Dog {
    fn fanno_verso(&self) {
        println!("Bau Bau");
    }
    fn si_cibano(&self) {
        println!("I cuccioli di cani sono allattati.");
    }
}

impl Uccelli for Crow {
    fn volano(&self) {
        println!("I corvi volano.");
    }
}

```



```

fn main() {
    let dog = Dog { nome: "Fido".to_string() };
    let corvo = Crow { weight: 2.20 };

    Mammiferi::fanno_verso(&dog);
    dog.si_cibano();
    corvo.fanno_verso();
    corvo.volano();
}

```

Funzioni con astrazione del parametro

lezione del 24 marzo

- Invece di dover scrivere esplicitamente il tipo concreto è possibile indicare solo il tratto che il tipo deve soddisfare
- Funzioni che accettano (come parametro) o restituiscono (come valore di ritorno) un tipo che implementa uno (o più con la notazione `+`) tratti specifici
- Quando il compilatore incontra una funzione con parametro `impl Trait`, sta generando una funzione generica **monomorfizzata**, ossia per ogni tipo concreto con cui la funzione viene chiamata, il compilatore crea una versione specializzata della funzione con quel tipo concreto specifico al posto di `impl Trait`

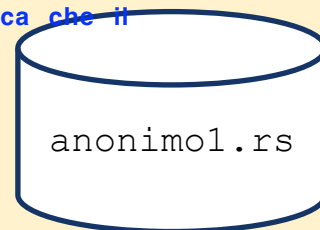
```
use std::fmt::Debug;
```

```
fn mostra_debug(valore: impl Debug) {  
    println!("Debug: {:?}", valore);  
}
```

quindi la

```
fn main() {  
    mostra_debug("ciao");  
    let my_vec = vec![1, 2, 3];  
    mostra_debug(my_vec);  
}
```

quando dichiaro il parametro `valore`, esso implementa il tratto `Debug`. Significa che il parametro deve necessariamente implementare il tratto `Debug`.



TIPO COMPLETO: tipo usato dal compilatore per costruire la procedura

se specifico il tratto ma non dettaglio il tipo, questo corrisponde ad una descrizione generica del tipo. Ci chiediamo quindi: qual è la regola che impone il compilatore, ovvero qual è il tipo? per rispondere, diciamo che il tipo deve implementare il tratto

MONORFIZZAZIONE : il compilatore costruisce una funzione per ogni tipo concreto che trova nel codice sorgente. Deve sempre essere noto il tipo quando costruisco l'eseguibile. Il compilatore genera una versione della funzione per il tipo str e una per il tipo vec, perchè sono i tipi che il compilatore ha trovato durante la compilazione. Vengono generate tante funzioni quante sono i tipi che il programmatore ha usato nel codice. Precisiamo anche che il compilatore ricicla parte delle repliche delle funzioni per questioni di ottimizzazione


```
trait Forma {
    fn area(&self) -> f64;
    fn perimetro(&self) -> f64;
}
```

```
struct Cerchio { raggio: f64}
```

```
impl Forma for Cerchio {
```

```
    fn area(&self) -> f64 {
        std::f64::consts::PI * self.raggio * self.raggio
    }
```

```
    fn perimetro(&self) -> f64 {
        2.0 * std::f64::consts::PI * self.raggio
    }
}
```

```
struct Rettangolo {
```

```
    larghezza: f64,
    altezza: f64,
}
```

```
impl Forma for Rettangolo {
```

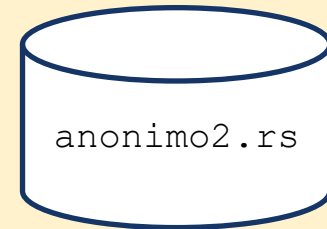
```
    fn area(&self) -> f64 {
        self.larghezza * self.altezza
    }
```

```
    fn perimetro(&self) -> f64 {
        2.0 * (self.larghezza + self.altezza)
    }
}
```

```
// Funzione che accetta qualsiasi tipo che implementa il tratto 'Forma'
```

```
fn stampa_dettagli_forma(forma: impl Forma) {
    println!("Area: {}", forma.area());
    println!("Perimetro: {}", forma.perimetro());
    println!("---");
}
```

```
fn main() {
    let cerchio = Cerchio { raggio: 5.0 };
    let rettangolo = Rettangolo { larghezza: 4.0, altezza: 6.0 };
    stampa_dettagli_forma(cerchio);
    stampa_dettagli_forma(rettangolo);
}
```



Funzioni con astrazione del valore di ritorno

```
trait Forma {
    fn area(&self) -> f64;
    fn descrizione(&self) -> &str;
}
struct Cerchio {
    raggio: f64,
}
struct Rettangolo {
    base: f64,
    altezza: f64,
}
impl Forma for Cerchio {
    fn area(&self) -> f64 {
        std::f64::consts::PI * self.raggio * self.raggio
    }
    fn descrizione(&self) -> &str { "Cerchio" }
}
impl Forma for Rettangolo {
    fn area(&self) -> f64 {
        self.base * self.altezza
    }
    fn descrizione(&self) -> &str { "Rettangolo" }
}
fn crea_forma_grande() -> impl Forma {
    Cerchio { raggio: 10.0 }
}
fn main() {
    let forma = crea_forma_grande();
    println!("Forma: {}", forma.descrizione());
    println!("Area: {:.2}", forma.area());
}
```

per completezza la notazione impl “tratto” si può usare per esprimere il valore di ritorno dal punto di vista sintattico



qui il valore di ritorno gode di un certo tratto, e dobbiamo assicurarci che il tipo implementi il tratto che scriv

Funzioni con astrazione del valore di ritorno

```
fn crea_forma_grande() -> impl Forma {  
    Cerchio { raggio: 10.0 }  
}
```

restituisco il valore di un tipo che implementa il tratto forma, ma non posso restituirne due perchè il compilatore deve conoscere la dimensione del tipo costituito, in quanto alloca dello spazio per il valore di ritorno

E se volessi restituire diversi (non uno solo) tipi che implementano il tratto Forma ?

```
fn crea_forma(grande: bool) -> impl Forma {  
    if grande {  
        Cerchio { raggio: 10.0 }  
    } else {  
        Rettangolo { base: 5.0, altezza: 3.0 }  
    }  
}
```



Serve un puntatore perchè quando il compilatore genera il codice non conosce la dimensione del dato e quindi mi serve un puntatore per avere un riferimento sia al dato, sia al tipo che mi permette di accedere al metodo specifico di implementazione per quel tipo. La variabile usata con l'oggetto tratto sarà sempre una variabile che usa un riferimento, che può essere un box o un rc... PER ACCEDERE ALL'OGGETTO TRATTO HO BISOGNO DI UN RIFERIMENTO

Invocare un tratto

- L'invocazione (**dispatching**) dei metodi di un tratto può avvenire in due modalità distinte
 - Invocazione **statica**: se il tipo del valore è noto, il compilatore può identificare l'indirizzo della funzione da chiamare e generare il codice corrispondente senza alcuna penalità
 - Invocazione **dinamica**: se si dispone di un **puntatore** ad un valore di cui il compilatore sa solo che implementa un dato tratto, occorre eseguire una chiamata indiretta, passando per una VTABLE
- Variabili o parametri destinati a contenere puntatori (riferimenti, Box, Rc, ...) ad un valore che implementa un tratto sono annotati con la parola chiave **dyn**

```
trait Print {  
    fn print(&self);  
}  
struct S { i: i32 }  
impl Print for S {  
    fn print(&self){  
        println!("S {}", self.i); }  
}
```

```
fn process(v: &dyn Print){  
    v.print();  
}  
  
fn main() {  
    process(&S{i: 0});  
}
```

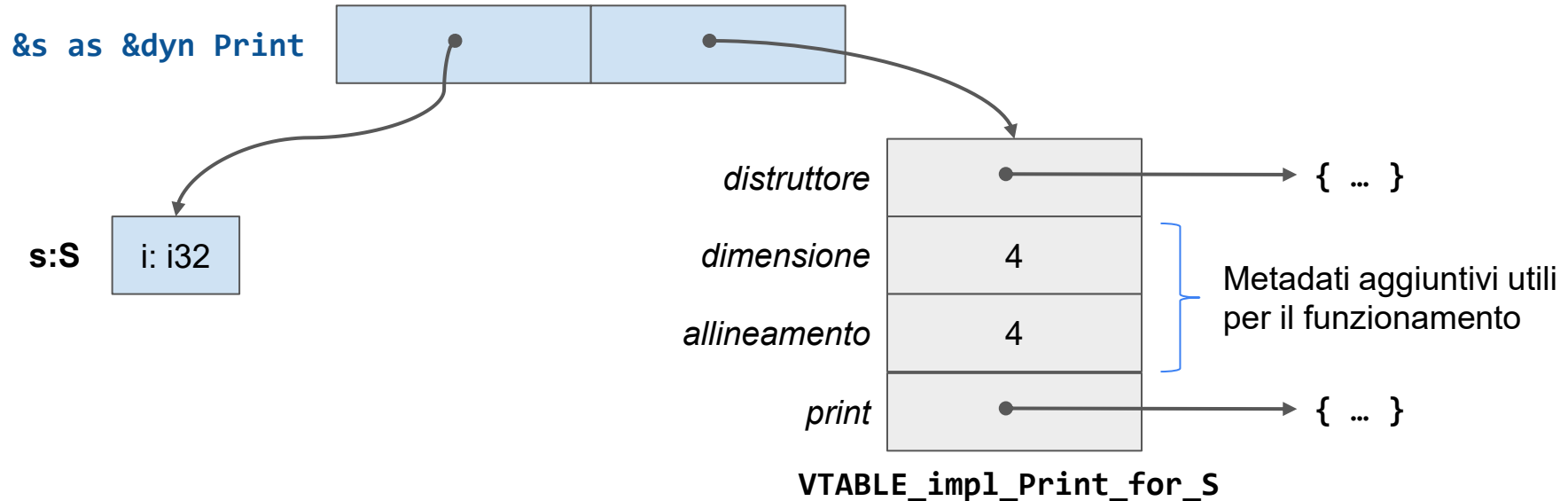
V è un riferimento generico ad un tipo che implementa il tratto Print. Ciò che è vincolante è il tratto

Serve un puntatore perchè quando il compilatore genera il codice non conosce la dimensione del dato e quindi mi serve un puntatore per avere un riferimento sia al dato, sia al tipo che mi permette di accedere al metodo specifico di implementazione per quel tipo. La variabile usata con l'oggetto tratto sarà sempre una variabile che usa un riferimento, che può essere un box o un rc... PER ACCEDERE ALL'OGGETTO TRATTO HO BISOGNO DI UN RIFERIMENTO

sono strutture che fanno riferimento ad un tratto e usando un puntatore doppio, ovvero è un fat pointer che contiene due puntatori: uno punta al dato, l'altro all'implementazione del tratto per quel tipo. la struttura dati prende il nome di VTABLE. Essa contiene i puntatori ai vari metodi che implementano il tratto e di default il metodo distruttore, più informazioni sulla dimensione del tipo e di allineamento nel caso di un array.

Oggetti-tratto

- I riferimenti/puntatori ai tipi tratto vengono detti oggetti-tratto
 - Possono essere condivisi o mutabili e devono rispettare le regole dell'esistenza in vita del valore a cui fanno riferimento
- Gli oggetti-tratto vengono implementati tramite *double pointer*
- Per ogni tratto che definisce oggetti-tratto sono allocate un numero di VTABLE pari al numero di tipi concreti che implementano il tratto



```

trait Shape {
    fn area(&self) -> f64; }
trait Drawable: Shape { fn draw(&self);}

struct Rectangle {
    width: f64,
    height: f64,
}
impl Shape for Rectangle {
    fn area(&self) -> f64 {self.width * self.height}
}

impl Drawable for Rectangle {
    fn draw(&self) {
        println!("Drawing a rectangle with width {} and height {}.",
            self.width, self.height);
    }
}

struct Square { side: f64, }
impl Shape for Square {
    fn area(&self) -> f64 { self.side * self.side }
}

impl Drawable for Square {
    fn draw(&self) {
        println!("Drawing a square with side {}.", self.side);
    }
}

```

qui c'è un dispatching dinamico.

qui ho solo una funzione che ha come parametro l'oggetto tratto, quindi non ho monorfizzazione. La VTABLE è specifica per l'implementazione di quel tratto per quel tipo. Quindi ci chiediamo, quante VTABLE ho nel programma? La risposta è 2 perchè ho due implementazioni di due tipi diversi per quel tratto

passo un riferimento

```

fn draw_shape(shape: &dyn Drawable) {
    shape.draw();
    println!("Area: {}", shape.area());
}

fn main() {
    let rectangle =
        Rectangle {width:3.0,height: 4.0};
    let square = Square { side: 2.0};

    draw_shape(&rectangle);
    draw_shape(&square);
}

```



```
trait Shape { fn area(&self) -> f64; }
```

```
struct Circle { radius: f64}
```

```
impl Shape for Circle {
```

```
    fn area(&self) -> f64 { std::f64::consts::PI * self.radius * self.radius }
```

```
}
```

```
struct Rectangle {
```

```
    width: f64,
```

```
    height: f64
```

```
}
```

```
impl Shape for Rectangle {
```

```
    fn area(&self) -> f64 { self.width * self.height }
```

```
}
```

questo vec memorizza oggetti.tratto, quindi memorizza riferimenti a dei tipi che implementano

tutti lo stesso tratto. Il vec contiene un riferimento ad un circle e ad un rettangolo. La struttura dati non esplicita il tipo ma solo il tratto.

```
fn main() {
```

```
    let circle = Circle { radius: 5.0 };
```

```
    let rectangle = Rectangle { width: 3.0, height: 4.0 };
```

```
    let mut shapes: Vec<&dyn Shape> = Vec::new();
```

```
    shapes.push(&circle);
```

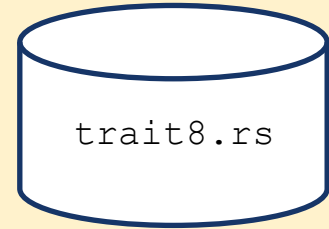
```
    shapes.push(&rectangle);
```

```
    for shape in shapes {
```

```
        println!("Area: {}", shape.area());
```

```
    }
```

```
}
```



Vettore di riferimenti a variabili di un tipo che implementa il tratto Shape

CI SONO VINCOLI:

1) IL COMPILATORE non deve avere la necessità di esplicitare la dimensione, lui deve solo usare riferimenti quando ci sono oggetti-tratto. I metodi devono SEMPRE E SOLO USARE RIFERIMENTI

Requisiti per gli oggetti-tratto

- I requisiti per un tratto per poter essere utilizzato come oggetto tratto deve avere tutte le firme dei metodi che soddisfano le seguenti condizioni:
 - Assenza di parametri di tipo generici (capiremo meglio dopo che cosa vuol dire questo)
 - Self non deve essere obbligatoriamente un tipo di dimensione statica e nota a livello di compilazione (**Sized**)
 - Self non deve essere usato per valore in nessun parametro, né per il valore di ritorno
- Un tratto può diventare oggetto-tratto se i suoi metodi possono essere chiamati senza la necessità di conoscere la dimensione statica specifica del tipo che implementa il tratto, questo è necessario perché gli oggetti tratto sono rappresentati esclusivamente da un puntatore al dato e da un puntatore alla VTABLE.

dato un tipo, quel tipo implementa Sized? Se il tratto è sized significa che per quel tipo la dimensione è nota. un i32 è di tipo sized I TIPI SONO DI NATURA SIZED.

Tuttavia ci sono situazioni in cui il vincolo non sussiste: per i tipi non sized, come gli slice, non si conosce la dimensione.

Se il tipo deve essere necessariamente di tipo Sized per un tratto, il tratto NON È COSTRUIBILE COME OGGETTO-TRATTO PERCHÈ NON POSSO USARE ESCLUSIVAMENTE I DUE PUNTATORI, MA HO NECESSITÀ DI CONOSCERE LA DIMENSIONE DEL TIPO.

SE IL TRATTO È SIZED ALLORA IMPEDISCO DI COSTRUIRE OGGETTI TRATTO PER QUEL TRATTO

Sized vs. ?Sized

- Un tipo è Sized se la sua dimensione è nota a livello di compilatore.
- La maggior parte dei tipi in Rust sono Sized, tra cui: tipi primitivi, array, puntatori.
- Alcuni tipi in Rust non hanno una dimensione nota al momento della compilazione. I più comuni sono: slice ([T]), oggetti tratto (dyn Trait) e string slice (str).
- Il vincolo : Sized è spesso implicito. Se non si specifica diversamente, Rust assume che i parametri di tipo siano Sized.
- Se si vuole operare con un tipo che può essere Unsized occorre utilizzare la notazione ?Sized.

```
fn print_maybe_unsized<T: ?Sized>(val: &T) {  
    println!("Valore (forse) unsized: {:?}" , val);  
}
```

- Se si vogliono accettare anche tipi non Sized, si deve rimuovere il vincolo e usare un puntatore.

```
struct MyUnsizeStruct<T: ?Sized> {  
    data: Box<T>,  
}
```

Poichè T potrebbe essere non Sized significa che non serve che il compilatore conosca la dimensione. Serve solo sapere che data è un box di qualcosa

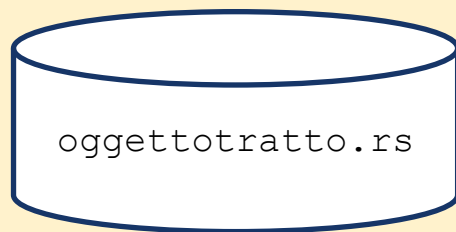
Altro uso degli Oggetti Tratto

12:10

- Collezioni di tipi eterogenei: Quando si deve lavorare con una collezione di valori di tipi diversi che condividono un comportamento comune (definito da un tratto).
- Ad esempio:
 - `Vec<Box<dyn Tratto>>` è una collezione di Box di valori di tipi diversi che condividono lo stesso tratto

```
trait Animale {  
    fn fai_verso(&self);  
}  
struct Cane {}  
impl Animale for Cane {  
    fn fai_verso(&self) {  
        println!("Bau!");  
    }  
}  
struct Gatto {}  
impl Animale for Gatto {  
    fn fai_verso(&self) {  
        println!("Miao!");  
    }  
}  
fn main() {  
    let animali: Vec<Box<dyn Animale>> = vec![Box::new(Cane {}), Box::new(Gatto {})];  
    for animale in animali {  
        animale.fai_verso();  
    }  
}
```

i dyn devono SEMPRE avere un puntatore: ci deve essere o un &dyn o un Box di dyn.



l'oggetto-tratto deve sempre avere un puntatore dietro

qui finisce la sezione sui tratti

Text

Tratti nella libreria standard

utilizza anche il libro per studiare questa parte

- Rust definisce un insieme di tratti base la cui eventuale implementazione da parte di un dato tipo abilita una serie di scorciatoie sintattiche legate agli operatori presenti nel linguaggio
 - Sostituendo, a tutti gli effetti, il meccanismo di **operator overloading** presente nel linguaggio C++
- E' possibile confrontare due istanze di un dato tipo con gli operatori **==** e **!=** se il tipo implementa i tratti **Eq** o **PartialEq**
 - E' possibile un confronto basato su **<**, **>**, **<=** e **>=** se il tipo implementa i tratti **Ord** o **PartialOrd**
- Gli operatori binari **+**, **-**, *****, **/**, **%**, **&**, **^**, **<<**, **>>** sono rispettivamente associati ai tratti **Add**, **Sub**, **Mul**, **Div**, **Rem**, **BitAnd**, **BitXor**, **Shl**, **Shr**
- Le operazioni unarie **-** e **!** corrispondono a **Neg** e **Not**

la libreria standard fornisce tratti standard sui tipi elementari

Dato un tipo composto, come una struct di interi, se voglio clonarla devo derivare clone

1) derivazione

2) implementazione dettagliata

se voglio fare implementazioni devo esplicitarle con derivazione

Definizioni dei tratti

per implementare il tratto per il tipo devo scrivere il metodo

```
trait Add<Rhs = Self> {  
    type Output;  
    fn add(self, rhs: Rhs) -> Self::Output;  
}
```

Il tipo Output definisce il tipo del risultato dell'operazione.

Il parametro generico Rhs (Right Hand Side) permette di definire operazioni tra tipi diversi (di default è Self).

dietro l'operatore + c'è il tratto add

```
trait Shl<Rhs> {  
    type Output;  
    fn shl(self, rhs: Rhs) -> Self::Output;  
}
```

Rhs è il tipo del secondo parametro
se scrivo <Rhs> effettuo lo shift a sinistra di una variabile di un certo numero di passi
se scrivo <Rhs>

Il tipo di Rhs (la quantità da shiftare) non è per default Self, ma dipende dal contesto (solitamente un tipo intero senza segno).


```
use std::ops::Add;
```

```
struct Point {  
    x: i32,  
    y: i32,  
}
```

il tratto standard va citato se voglio fare un'implementazione personalizzata del tratto

```
impl Add for Point {  
    type Output = Self;
```

p1 in questo caso implementa il tratto Add per il tipo Point

definisco un tipo attributo type output di tipo Self

```
    fn add(self, other: Self) -> Self::Output {  
        Point {  
            x: self.x + other.x,  
            y: self.y + other.y,  
        }  
    }  
}
```

Text
Text

devo sempre dire cosa è Output perchè viene specificato dalla definizione del tratto

```
fn main() {  
    let p1 = Point { x: 1, y: 0 };  
    let p2 = Point { x: 2, y: 3 };  
    let p3 = p1 + p2; // Usa l'implementazione di Add  
    println!("p3.x: {}, p3.y: {}", p3.x, p3.y);  
}
```



Gestire i confronti di uguaglianza

Per fare l'uguaglianza i tratti devono godere delle proprietà di riflessi

- Implementare un confronto di uguaglianza richiede il puntuale rispetto di alcune proprietà logiche essenziali
 - Riflessività, se un oggetto è uguale a se stesso
 - Simmetria, se l'oggetto A è uguale all'oggetto B possiamo dire che l'oggetto B è uguale all'oggetto A
 - Transitività, se l'oggetto A è uguale all'oggetto B e l'oggetto B è uguale all'oggetto C, possiamo dire che l'oggetto A è uguale all'oggetto C
- Rust introduce due tratti per esprimere questa capacità
 - **PartialEq** ed **Eq**, entrambi definiti dai metodi **eq(&self, other: &RHS)-> bool** e **ne(&self, other: &RHS)-> bool**
 - **PartialEq** richiede che siano garantite, dall'implementazione, sia la proprietà simmetrica sia quella transitiva
 - **Eq** (che è un sotto-tratto di **PartialEq**) impone anche il rispetto della proprietà riflessiva: i tipi floating-point (f32 e f64) NON implementano questo tratto (NaN != NaN)
 - Il tratto **Eq** è un tratto senza metodi (tratto marker) e la sua dichiarazione solo per specificare che l'uguaglianza gode anche della proprietà **riflessiva**
- Il tipo associato **RHS (Right Hand Side)** definito, per default, come **Self**
 - In rari casi può essere necessario permettere confronti tra tipi differenti
 - Il metodo **ne(...)** è normalmente preso dalla sua implementazione di default, come opposto del risultato di **eq(...)**

Definizioni dei tratti `PartialEq` ed `Eq`

```
trait PartialEq <Rhs = Self>
where Rhs: ?Sized,
{
    fn eq(&self, other: &Rhs) -> bool;
    fn ne(&self, other: &Rhs) -> bool {
        !self.eq(other)
    }
}
```

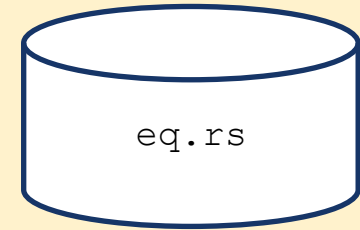
in `partialeq` confrontiamo due valori come riferimenti (e quindi non consumo il tipo), quindi c'è il discorso del right hand side.
i tratti marker sono quelli che non hanno metodi. Danno informazioni su una proprietà in particolare che per quel tipo si assicura la riflessività. Non può esistere il tratto `Eq` se non implemento il `PartialEq`.

```
trait Eq: PartialEq <Self> { }
```

Il parametro generico `Rhs` permette di effettuare un confronto tra tipi diversi (di default è `Self`) e può essere un tipo di dimensione ignota al compilatore.

Il tipo di `Rhs` deve essere `Self`, per rispettare il vincolo di riflessività

```
struct Point {  
    x: i32,  
    y: i32,  
}  
  
impl PartialEq for Point {  
    fn eq(&self, other: &Self) -> bool {  
        self.x == other.x && self.y == other.y  
    }  
}  
  
impl Eq for Point {}  
  
fn main() {  
    let point1 = Point { x: 1, y: 2 };  
    let point2 = Point { x: 1, y: 2 };  
    let point3 = Point { x: 3, y: 4 };  
  
    println!("point1 == point2: {}", point1 == point2); // true  
    println!("point1 == point3: {}", point1 == point3); // false  
}
```



Derivare metodi automaticamente

- Sebbene sia possibile implementare a mano tutti i metodi richiesti da un certo tratto, a volte è meglio affidarsi al compilatore
 - Se la definizione di una struct o di una enum è preceduta dall'attributo `#[derive(...)]`, il compilatore provvede ad aggiungere, automaticamente, l'implementazione dei tratti indicati all'interno del costrutto `derive(...)`
 - Solo un certo sottoinsieme di tratti possono essere generati automaticamente, spesso a condizione che i tipi dei dati contenuti nel tipo per cui si esegue la derivazione soddisfino opportuni vincoli (come, ad esempio, implementare a propria volta il tratto)

```
#[derive(PartialEq)]
struct Foo<T> {
    a: i32,
    b: T,
}
```

```
impl<T: PartialEq> PartialEq for Foo<T> {
    fn eq(&self, other: &Foo<T>) -> bool {
        self.a == other.a && self.b == other.b
    }

    fn ne(&self, other: &Foo<T>) -> bool {
        self.a != other.a || self.b != other.b
    }
}
```

qui non dovrei mettere b come numero i32?

```
#[derive(PartialEq, Eq)]
struct Point {
    x: i32,
    y: i32,
}
```

2 Point sono uguali se tutti i corrispondenti
valori dei loro campi sono uguali

```
fn main() {
    let point1 = Point { x: 1, y: 2 };
    let point2 = Point { x: 1, y: 2 };
    let point3 = Point { x: 3, y: 4 };

    println!("point1 == point2: {}", point1 == point2); // true
    println!("point1 == point3: {}", point1 == point3); // false
}
```

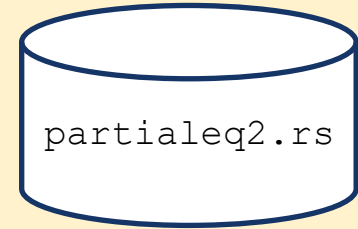
eq2.rs

```
#[derive(PartialEq)]
```

```
struct FloatingPointNumber {  
    value: f64,  
}
```

Text

```
fn main() {  
    let a = FloatingPointNumber { value: 1.0 };  
    let b = FloatingPointNumber { value: 1.01 };  
    let c = FloatingPointNumber { value: 1.000000001 };  
    let d = FloatingPointNumber { value: 1.000000001 };  
  
    println!("a == b: {}", a == b); // false  
    println!("a == c: {}", a == c); // false  
    println!("c == d: {}", c == d); // true  
}
```



```
struct FloatingPointNumber {
    value: f64,
}

impl PartialEq for FloatingPointNumber {
    fn eq(&self, other: &Self) -> bool {
        let epsilon = 0.00001;
        let difference = (self.value - other.value).abs();
        difference <= epsilon
    }
}

fn main() {
    let a = FloatingPointNumber { value: 1.0 };
    let b = FloatingPointNumber { value: 1.01 };
    let c = FloatingPointNumber { value: 1.000000001 };

    println!("a == b: {}", a == b); // false
    println!("a == b: {}", a == c); // true
}
```



Tipo associato Rhs (Right Hand Side) diverso da Self

```
struct Point {  
    x: i32,  
    y: i32,  
}  
  
impl PartialEq<i32> for Point {  
    fn eq(&self, other: &i32) -> bool {  
        self.x == *other && self.y == *other  
    }  
}  
  
fn main() {  
    let point = Point { x: 5, y: 5 };  
  
    // Confronto tra un punto e un valore i32  
    println!("point == 5: {}", point == 5); // true  
}
```

tra parentesi angolari specifico il right and side,



per poter implementare il `partial ord` devo necessariamente implementare il `partial eq` perchè l'ugualianza è un concetto necessario per poter fare un ordinamento

Gestire i confronti di ordine

il `partial order` restituisce un tipo `option`. Nel caso di `None` per una coppia di oggetti significa che non è possibile stabilire un ordinamento —> questo è il concetto di `partial ord`

- I tratti `PartialOrd` e `Ord` permettono rispettivamente di definire relazioni d'ordine parziali e totali su un dato insieme di valori

```
enum Ordering {  
  Less,  
  Equal,  
  Greater,  
}
```

```
trait PartialOrd<Rhs = Self>: PartialEq<Rhs>  
where Rhs: ?Sized, {
```

```
  fn partial_cmp(&self, other: &Rhs) ->
```

`Option<Ordering>;`

```
  // metodi con implementazione di default
```

```
  fn lt(&self, other: &Rhs) -> bool;  
  fn le(&self, other: &Rhs) -> bool;  
  fn gt(&self, other: &Rhs) -> bool;  
  fn ge(&self, other: &Rhs) -> bool;
```

```
}
```

Tipo `Rhs` non deve avere necessariamente una dimensione nota a livello di compilazione

```
trait Ord: Eq + PartialOrd<Self> {  
  fn cmp(&self, other: &Self)  
    -> Ordering;
```

```
  // implementazione di default  
  fn max(self, other: Self) -> Self;  
  fn min(self, other: Self) -> Self;  
}
```

è possibile confrontare due valori per determinare se uno è minore di, maggiore di, o uguale all'altro, ma non è garantito che tutti i valori possano essere confrontati tra loro.

Gli altri metodi (`lt`, `le`, `gt`, `ge`) forniscono implementazioni predefinite basate sul risultato di `partial_cmp`

Ordinamento parziale

l'implementazione del `partial_cmp` stabilisce il criterio di orientamento tra due elementi

partialord1.rs

in questo codice è noto il `partialEq`, ma non il `partialOrd`.
Per cui devo implementare il tratto `PartialOrd`

```
#[derive(Debug, PartialEq)]
struct Point {
    x: f64,
    y: f64,
}

impl PartialOrd for Point {
    fn partial_cmp(&self, other: &Self) -> Option<std::cmp::Ordering> {
        if self.x < other.x && self.y < other.y {
            Some(std::cmp::Ordering::Less)
        } else if self.x > other.x && self.y > other.y {
            Some(std::cmp::Ordering::Greater)
        } else if self.x == other.x && self.y == other.y {
            Some(std::cmp::Ordering::Equal)
        } else { None // In caso di confronto non definito }
    }
}

fn confronta (p1: &Point, p2: &Point) {
    match p1.partial_cmp(&p2) {
        Some(ordering) => {
            println!("{:?} è {:?} di {:?}", p1, ordering, p2);
        }
        None => {
            println!("Confronto non definito tra {:?} e {:?}", p1, p2);
        }
    }
}
```

```
fn main() {
    let point1 = Point { x: 1.0, y: 4.0 };
    let point2 = Point { x: 3.0, y: 2.0 };
    let point3 = Point { x: 10.0, y: 5.0 };

    confronta(&point1, &point2);
    confronta(&point1, &point3);
}
```

```
#[derive(Debug, Eq, PartialEq, PartialOrd)]
```

Ordinamento totale

```
struct Point {  
    x: i32,  
    y: i32,  
}
```

```
impl Ord for Point {  
    fn cmp(&self, other: &Self) -> std::cmp::Ordering {  
        if self.x != other.x {  
            self.x.cmp(&other.x) // ci si riconduce ad un confronto tra  
                                // valori di tipo intero che implementano il  
                                // tratto Ord  
        } else {  
            self.y.cmp(&other.y)  
        }  
    }  
}
```

il cmp deve fornire sempre una risposta.
prendo self con other, verifico se le due x sono diverse. Se sono diverse confronto le x. Applico quindi la cmp sulle x.

```
fn main() {  
    let point1 = Point { x: 3, y: 5 };  
    let point2 = Point { x: 3, y: 8 };  
  
    let max_point = point1.max(point2);  
    println!("Il punto con coordinate massime è {:?}", max_point);  
}
```



```

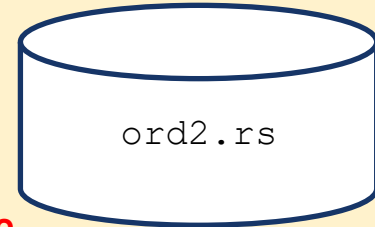
#[derive(Debug, Eq, PartialEq, PartialOrd)]
struct Person {
    name: String,
    age: i32,
}

impl Ord for Person {
    fn cmp(&self, other: &Self) -> std::cmp::Ordering {
        self.age.cmp(&other.age) // ci si riconduce al confronto
                                // tra i32 che implementano il tratto Ord
    }
}

fn main() {
    let john = Person { name: String::from("John"), age: 25 };
    let emma = Person { name: String::from("Emma"), age: 23 };
    let peter = Person { name: String::from("Peter"), age: 25 };

    println!("John is {:?}, compared to Emma.", john.cmp(&emma));
    // Stampa: John is Greater, compared to Emma.
    println!("John is {:?}, compared to Peter.", john.cmp(&peter));
    // Stampa: John is Equal, compared to Peter.
}

```



Indicizzare una struttura dati

- E' possibile utilizzare **sintatticamente** una struttura dati come fosse un array, abilitando la notazione **`t[i]`** se si implementano i tratti **`Index`** e **`IndexMut`**
 - L'espressione **`t[i]`** viene riscritta dal compilatore come **`*t.index(i)`**, se si accede in lettura al risultato dell'espressione, o come **`*t.index_mut(i)`**, se si accede in scrittura
- Questi due tratti sono definiti nel seguente modo

```
trait Index<Idx> {  
    type Output: ?Sized;  
    fn index(&self, index: Idx) -> &Self::Output;  
}  
  
trait IndexMut<Idx>: Index<Idx> {  
    fn index_mut(&mut self, index: Idx) -> &mut Self::Output;  
}
```

Indicizzare una struttura dati

- Molti tipi della libreria standard implementano i tratti `Index` e `IndexMut`
 - Poiché sia l'indice usato che il tipo associato restituito dall'operazione di indicizzazione sono parametrizzati, è lecito avere implementazioni multiple del tratto per un dato tipo
 - `Vec<T>`, ad esempio, consente di definire
 - indici numerici (`usize`) che restituiscono valori di tipo `T`
 - indici di tipo intervallo (`Range`) che restituiscono `Slice<T>`

```
// Vec<i32> implementa Index<usize, Output = i32>
let vec = vec![1, 2, 3, 4, 5];
let num: i32 = vec[0];
let num_ref: &i32 = &vec[0];

// Ma implementa anche Index<Range<usize>, Output=[i32]>
assert_eq!(&vec[1..4], &[2, 3, 4]);
```

```
use std::ops::{Index, IndexMut};
```

```
#[derive(Debug)]
struct StringSet {
    data: Vec<String>,
}
```



```
impl StringSet {
    fn new() -> Self {
        StringSet { data: Vec::new() }
    }
    fn add(&mut self, s: String) {self.data.push(s)}
}
```

Index: usize

```
impl Index<usize> for StringSet {
    type Output = String;
    fn index(&self, index: usize) -> &Self::Output {
        &self.data[index]
    }
}
```

Output: String

```
impl IndexMut<usize> for StringSet {
    fn index_mut(&mut self, index: usize) -> &mut Self::Output {
        &mut self.data[index]
    }
}
```

```
fn main() {
    let mut string_set=StringSet::new();
    string_set.add("apple".to_string());
    string_set.add("banana".to_string());
    string_set.add("orange".to_string());
    println!("Original {:?}",string_set);

    string_set[1] = "grape".to_string();
    println!("Modified {:?}",string_set);
}
```



```

use std::ops::Index;
struct MyVec{
    data: Vec<i32>,
    year: i32
}
impl MyVec {
    fn new(data: Vec<i32>) -> Self {
        MyVec{data: data, year: 2024}
    }
}

impl Index<usize> for MyVec {
    type Output = i32;

    fn index(&self, index: usize) -> &Self::Output {
        &self.data[index]
    }
}

fn main() {
    let my_vec = MyVec::new(vec![42, 2, 3, 4, 5]);

    // Accesso al campo data tramite indice
    println!("Primo: {:?}", my_vec[0]); // Stampato: Primo elemento: [42]
}

```

Index: usize

Output: i32



```

use std::ops::Index;
struct MyVec{
    data: Vec<i32>,
    year: i32
}
impl MyVec {
    fn new(data: Vec<i32>) -> Self {
        MyVec{data: data, year: 2024}
    }
}
impl Index<std::ops::Range<usize>> for MyVec {
    type Output = [i32];
    fn index(&self, index: std::ops::Range<usize>) -> &Self::Output {
        &self.data[index]
    }
}
fn main() {
    let my_vec = MyVec::new(vec![1, 2, 3, 4, 5]);

    // Accesso tramite intervallo di indici
    let slice = &my_vec[1..4];
    println!("Slice: {:?}", slice); // Stampato: [2, 3, 4]
}

```

Index: un
range

Output: uno slice

index3.rs

Conversione tra tipi

- I tratti **From** e **Into** permettono di effettuare conversioni di tipo, prendono possesso del valore lo convertono e ritornano il possesso al chiamante
 - I tratti sono perfettamente duali, scrivere **T: From<i32>** equivale a scrivere **i32: Into<T>**
 - Se si implementa il tratto **From**, il tratto **Into** viene generato automaticamente
 - L'implementazione del tratto **From** non è simmetrica: se è possibile passare dal tipo T al tipo U, non è affatto detto che sia possibile tornare indietro dal tipo U al tipo T

```
trait From<T>: Sized {  
    fn from(other: T) -> Self;  
}  
  
trait Into<T>: Sized {  
    fn into(self) -> T;  
}
```

Conversione tra tipi

```
#[derive(Debug)]
struct Point {
    x: i32,
    y: i32,
}

impl From<(i32, i32)> for Point {
    fn from((x, y): (i32, i32)) -> Self {
        Point { x, y }
    }
}

impl From<[i32; 2]> for Point {
    fn from([x, y]: [i32; 2]) -> Self {
        Point { x, y }
    }
}
```



```
fn main() {
    // from
    let p1 = Point::from((3, 1));
    let p2 = Point::from([5, 2]);

    // into
    let p3: Point = (1, 3).into();
    let p4: Point = [4, 0].into();

    println!("p1 {:?}", p1);
    println!("p2 {:?}", p2);
    println!("p3 {:?}", p3);
    println!("p4 {:?}", p4);

    // ERRORE! non vale la simmetria
    //let a1 = <[i32; 2]>::from(point);
    //let a2: [i32; 2] = point.into();
    //let t1 = <(i32, i32)>::from(point);
    //let t2: (i32, i32) = point.into();
}
```

da tupla a Point

da tupla a Point

```

#[derive(Debug)]
struct Point {
    x: i32,
    y: i32, }

impl From<(i32, i32)> for Point {
    fn from((x, y): (i32, i32)) -> Self { Point { x, y } }
}

impl From<Point> for (i32, i32) {
    fn from(Point { x, y }: Point) -> Self { (x, y) }
}

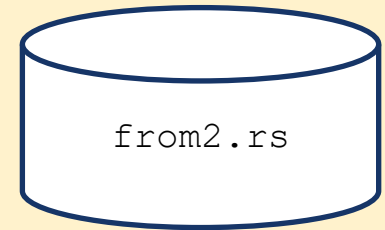
impl From<[i32; 2]> for Point {
    fn from([x, y]: [i32; 2]) -> Self { Point { x, y } }
}

impl From<Point> for [i32; 2] {
    fn from(Point { x, y }: Point) -> Self { [x, y] }
}

fn main () {
    let point = Point::from((0, 0));
    let point: Point = (0, 0).into();
    let tuple: (i32, i32) = point.into();
    let point = Point::from([1, 1]);
    let point: Point = [0, 0].into();
    let array = <[i32; 2]>::from((2,2));

    println!(" {:?} {:?} {:?} ", point, tuple, array);
}

```



Conversione tra tipi

- Per gestire le conversioni tra tipi che possono fallire vengono forniti i tratti **TryFrom** e **TryInto** che posseggono le stesse proprietà di **From** e **Into**
 - I metodi **try_from** e **try_into** ritornano il tipo **Result<T,E>** per garantire la gestione dell'errore

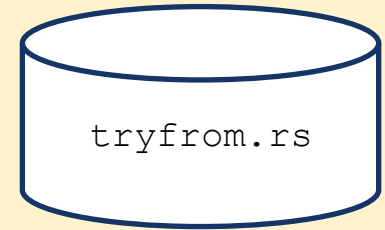
```
pub trait TryFrom<T>: Sized {  
    type Error;  
    fn try_from(value: T) -> Result<Self, Self::Error>;  
}  
  
pub trait TryInto<T>: Sized {  
    type Error;  
    fn try_into(self) -> Result<T, Self::Error>;  
}
```

```
use std::convert::TryFrom;
struct EvenNumber(i32);
impl TryFrom<i32> for EvenNumber {
    type Error = &'static str;

    fn try_from(value: i32) -> Result<Self, Self::Error> {
        if value % 2 == 0 {
            Ok(EvenNumber(value))
        } else {
            Err("il numero non è pari.")
        }
    }
}

fn verifica (value: i32, result: Result<EvenNumber, &str>)
{
    match result {
        Ok(even) => println!("Numero pari: {}", even.0),
        Err(err) => println!("Errore: ho ricevuto {}, ma {}", value, err),
    }
}

fn main() {
    let even_number = 42;
    let result = EvenNumber::try_from(even_number);
    verifica(even_number, result);
    let odd_number = 37;
    let result_odd = EvenNumber::try_from(odd_number);
    verifica(odd_number, result_odd);
}
```



Conversione tra tipi

- Il tratto **FromStr** permette di gestire la conversione da stringa e l'eventuale fallimento
 - Il metodo **from_str()** viene implicitamente richiamato tutte le volte che si utilizza il metodo **parse()**

```
pub trait FromStr {  
    type Err;  
    fn from_str(s: &str) -> Result<Self, Self::Err>;  
}
```



```
use std::num::ParseIntError;
```

```
fn verifica (value: &str, result: Result<i32, ParseIntError> )
```

```
{
```

```
    match result {
```

```
        Ok(num) => println!("Numero: {}", num),
```

```
        Err(err) => println!("Errore: ho ricevuto {} {}", value, err),
```

```
    }
```

```
}
```

```
fn main() {
```

```
    // Utilizziamo il metodo parse() fornito dal tratto FromStr
```

```
    // per convertire la stringa in un numero intero
```

```
    let number_str = "42";
```

```
    let number = number_str.parse(); // il valore di ritorno di parse()
```

```
    // è di tipo Result<i32, ParseIntError>
```

```
    verifica(number_str, number);
```

```
    let number_str2 = "quarantadue";
```

```
    let number2= number_str2.parse();
```

```
    verifica(number_str2, number2);
```

```
}
```

fromstr1.rs

Dereferenziare un valore

- E' possibile trattare una struttura dati come se fosse sintatticamente un puntatore implementando i tratti **Deref** e **DerefMut**
 - La sintassi ***t** per un tipo che implementa **Deref** e che non sia un riferimento né un puntatore nativo equivale a ***(t.deref())** e restituisce un valore immutabile di tipo **Self::Target**
 - Analogamente, se il tratto implementa **DerefMut**, ***t** equivale a ***(t.deref_mut())** e restituisce un valore mutabile di tipo **Self::Target**

```
trait Deref {  
    type Target: ?Sized;  
    fn deref(&self) -> &Self::Target;  
}  
  
trait DerefMut: Deref {  
    fn deref_mut(&mut self) -> &mut Self::Target;  
}
```



Dereferenziare un valore

- Il tipo `Self::Target` deve essere qualcosa contenuto, posseduto o referenziato in `Self`
- La notazione `*t` prende a prestito, in modo condiviso o esclusivo, il valore `t` per poter eseguire il metodo del tratto corrispondente
 - I riferimenti restituiti da `deref(&self)` o da `deref_mut(&mut self)` prolungano, come conseguenza delle regole sul tempo di vita, il prestito di `self` fino al termine del loro utilizzo

Dereferenziare un valore

```
struct Selector {  
    elements: Vec<String>,  
    current: usize  
}  
  
use std::ops::Deref;  
  
impl Deref for Selector {  
    type Target = String;  
    fn deref(&self) -> &Self::Target  
        { & self.elements[ self.current ] }  
}  
  
let mut s = Selector{elements: vec!["a".to_string(), "b".to_string()], current:0};  
assert_eq!(*s, "a");  
  
s.current = 1;  
assert_eq!(*s, "b");
```



```
use std::ops::Deref;
#[derive(Debug)]
struct CustomStruct {
    number: i32,
    boxed_value: Box<i32>,
}

impl Deref for CustomStruct {
    type Target = i32;

    fn deref(&self) -> &Self::Target {
        &self.number
    }
}

fn main() {
    let boxed_value = Box::new(42);
    let custom_struct = CustomStruct { number: 10, boxed_value };

    println!("Dereferencing custom_struct: {:?}", *custom_struct);
    // Prints: Dereferencing custom_struct: 10
}
```



Deref Coercion

- I due tratti svolgono anche un secondo ruolo: permettono la **conversione automatica** dal tipo `&Self` al tipo `&Self::Target` e da `&mut Self` a `&mut Self::Target`.
- Questa proprietà viene detta **deref coercion**
- Si applica a:
 - smart pointer (puntatori che puntano ad un dato nello heap), per default
 - tipi wrapper (tipi che incapsulano un dato), se implementano il tratto `Deref`
- Quando il compilatore Rust si aspetta un riferimento di tipo `&U` (o `&mut U`) ma riceve un `&T` (o `&mut T`), esso verifica se `T` implementa `Deref<Target = U>` (o `DerefMut<Target = U>`). Se lo fa, il compilatore inserisce implicitamente una o più chiamate al metodo `deref` (o `deref_mut`) per ottenere il riferimento desiderato.

Deref Coercion

- Nel caso di tipo puntatore ad un dato nello heap il compilatore esegue la deref coercion, chiamando il metodo `deref` e restituendo il tipo `Target`.

```
fn main() {  
    let b:Box<i32> = Box::new(10);  
    println!("{}", b);  
}
```



```
struct CustomStruct {  
    number: i32,  
    boxed_value: Box<i32>,  
}  
fn main() {  
    let boxed_value = Box::new(42);  
    let custom_struct = CustomStruct { number: 10, boxed_value };  
    println!("Dereferencing boxed_value: {:?}", custom_struct.boxed_value);  
}
```



Deref Coercion

- Permette di utilizzare i tipi in modo più flessibile
- Tramite la deref coercion un `&String` viene automaticamente convertito in `&str`
- Si possono così applicare alle `String` tutti i metodi che lavorano su un `&str`

```
fn main() {  
    let owned_string = String::from("Hello, world!");  
  
    // Funzione che accetta un &str  
    fn print_str(s: &str) {  
        println!("{}", s);  
    }  
  
    // La String viene automaticamente dereferenziata in &str  
    print_str(&owned_string); // Qui avviene la Deref Coercion  
    // è come se avessimo fatto: print_str(owned_string.deref());  
  
    let string_slice = &owned_string; // Altro esempio di Deref Coercion: &String => &str  
    println!("Lunghezza della stringa {}", string_slice.len());  
    println!("Prime 5 lettere: {}", &owned_string[0..5]);  
    println!("Conversione in maiuscolo: {}", string_slice.to_uppercase());  
    println!("Stringa sostituita: {}", string_slice.replace("world", "Rust"));  
}
```




```
use std::ops::Deref;

struct MyData {value: i32}

struct Container {data: Box<MyData>}

impl Deref for Container {
    type Target = MyData;

    fn deref(&self) -> &Self::Target {
        &self.data
    }
}

fn main() {
    let my_data = MyData { value: 42 };

    let container = Container { data: Box::new(my_data) };

    // dereferenziazione automatica per accedere al valore all'interno di Container
    println!("Value inside Container: {}", container.value);
}
```



Visualizzare i contenuti

- Macro come **println!** e **format!** consentono di stampare un valore associato al segnaposto **{}** a condizione che tale valore implementi il tratto **Display**
 - Tale tratto rappresenta la capacità del tipo di creare una visualizzazione di un proprio valore comprensibile ad un **utente finale**
 - Da un punto di vista pratico, non può essere derivato
 - La sua implementazione richiede la definizione di un solo metodo

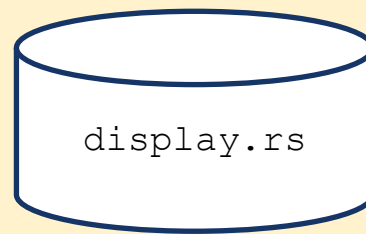
```
trait Display {  
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result;  
}
```

- Il tratto **Debug** ha la stessa firma di **Display**
 - Il suo scopo, tuttavia, è differente: creare una rappresentazione comprensibile al programmatore del valore
 - Viene attivato usando il segnaposto **{:?}**
 - Può essere sintetizzato automaticamente attraverso l'annotazione **#[derive(Debug)]**

```

#[derive(Debug)]
struct Libro {
    titolo: String,
    autore: String,
    anno: u32,
}
// Implementazione del tratto Display
impl fmt::Display for Libro {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> fmt::Result {
        write!(f,
            "Libro: \"{}\" di {} ({}\"",
            self.titolo,
            self.autore,
            self.anno
        )
    }
}
fn main() {
    let mio_libro = Libro {
        titolo: "Rust Programming".to_string(),
        autore: "Steve Klabnik".to_string(),
        anno: 2023,
    };
    println!("{}", mio_libro); // Stampa utilizzando Display
    println!("{:?}", mio_libro); // Stampa utilizzando Debug
}

```



La macro `write!` serve per scrivere testo formattato in un buffer o in qualsiasi destinazione che implementi il trait `std::fmt::Write`.

Libro: "Rust Programming" di Steve Klabnik (2023)
 Libro { titolo: "Rust Programming", autore: "Steve Klabnik", anno: 2023 }

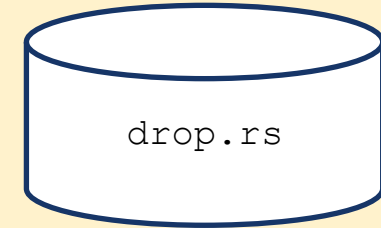
Copia e duplicazione

- Il tratto **Clone** indica la capacità di un tipo di creare un duplicato di un valore dato
 - Questo può essere usato per trasformare un riferimento (&T) in un valore posseduto (T)
 - Tale trasformazione, ovviamente, può essere molto costosa in termini di tempo e memoria, tuttavia è sempre sotto il controllo del programmatore perché tale funzionalità non è mai attivata automaticamente
- Il tratto **Copy** è un sotto-tratto di **Clone**
 - Esso implica il fatto che la copia ottenuta sia - a tutti gli effetti - il solo duplicato dei bit presenti nel valore originale, senza nessuna trasformazione
 - La sua implementazione può essere veloce ed efficiente (si basa sulla funzione `memcpy(...)`)
 - Questo tratto può essere implementato solo da strutture dati i cui campi implementino il tratto stesso
- La presenza del tratto **Copy** trasforma la semantica delle operazioni di assegnazione
 - Quello che normalmente determina un **movimento**, che rende inaccessibile il valore originale, diventa una **copia**
 - **Copy** è in mutua esclusione con **Drop**

Rilasciare le risorse

- Il tratto **Drop** permette di definire il metodo **drop(&mut self)** che fa le veci del distruttore nel linguaggio C++
 - Il compilatore Rust garantisce che tale metodo sarà chiamato nel momento in cui la variabile che contiene un valore di questo tipo uscirà dal proprio scope sintattico o subito prima che le venga assegnato un nuovo valore
 - Questo permette di rilasciare eventuali risorse possedute dalla struttura dati o abilitare comportamenti duali a quelli della creazione dell'oggetto, tipici del pattern RAII - Resource Acquisition Is Initialization
- Questo tratto è mutuamente esclusivo con il tratto **Copy**
 - Questo vincolo elimina, in assenza di blocchi **unsafe {...}**, la possibilità di avere un doppio rilascio delle risorse presenti sullo heap
- Questo tratto si accompagna alla funzione globale **fn drop<T>(_x:T) {}**
 - Essa forza il passaggio di possesso di un valore ad una nuova variabile (_x) che uscirà di scena subito, provocandone la distruzione

```
fn main() {  
    let vec = vec![1, 2, 3];  
  
    println!("Prima di drop: {:?}" , vec);  
  
    drop(vec); // Qui il vettore viene distrutto.  
  
    // println!("Dopo drop: {:?}" , vec); // Questo causerebbe un errore di compilazione!  
  
    println!("La memoria di 'vec' è stata liberata.");  
}
```



Definire un intervallo

- Attraverso l'utilizzo dell'operatore `..` è possibile definire intervalli di valori per tutti i tipi che implementano il tratto **`RangeBounds<T>`**
 - Quest'ultimo è implementato da diversi tipi base in Rust e consente l'utilizzo di sintassi come `..`, `a..`, `..b`, `..=c`, `d..e`, `f..=g`
 - E' necessario implementare i metodi **`end_bound(&self)`** e **`start_bound(&self)`** che ritornano entrambi il tipo **`Bound<&T>`**

```
pub trait RangeBounds<T>{  
    fn start_bound(&self) -> Bound<&T>;  
    fn end_bound(&self) -> Bound<&T>;  
    fn contains<U>(&self, item: &U) -> bool { ... }  
}
```

```
pub enum Bound<V>{  
    Included(V),  
    Excluded(V),  
    Unbounded,  
}
```

```
use std::ops::RangeBounds;
fn main() {
    let range = 1..; // Definizione di un range
    let end = range.end_bound(); // Ottenere il limite superiore del range

    match end {
        std::ops::Bound::Included(&upper) => println!("Limite superiore incluso:{}", upper),
        std::ops::Bound::Excluded(&upper) => println!("Limite superiore escluso:{}", upper),
        std::ops::Bound::Unbounded => println!("Il range non ha limite superiore"),
    }
}
```




```
use std::ops::{Bound, RangeBounds};
```

```
struct MyRange<T> {  
    start: T,  
    end: T,  
}
```

```
impl<T> RangeBounds<T> for MyRange<T> where T: Copy {  
    fn start_bound(&self) -> Bound<&T> {  
        Bound::Included(&self.start)  
    }  
    fn end_bound(&self) -> Bound<&T> {  
        Bound::Excluded(&self.end)  
    }  
}
```

```
fn main() {  
    let my_range = MyRange { start: 1, end: 5 };
```

```
    match my_range.end_bound() {  
        Bound::Included(&upper) => println!("Il limite superiore è incluso: {}", upper),  
        Bound::Excluded(&upper) => println!("Il limite superiore è escluso: {}", upper),  
        Bound::Unbounded => println!("Il range non ha limite superiore"),  
    }  
}
```



```
fn main() {  
    let range = 0..10;  
  
    println!("Does the range 0..10 contain 5? {}", range.contains(&5)); // true  
    println!("Does the range 0..10 contain 10? {}", range.contains(&10)); // false  
  
    let inclusive_range = 0..=10;  
    println!("Does the inclusive range 0..=10 contain 10? {}", inclusive_range.contains(&10));  
    // true  
}
```



Descrivere un errore

- In Rust gli errori vengono ritornati attraverso l'utilizzo del tipo **Result<T,E>**
- Il tipo generico **E** può assumere qualsiasi valore tuttavia è preferibile utilizzare solo tipi che implementano il tratto **Error**
- Per implementare **Error**, un tipo deve semplicemente implementare il tratto **Display** (dal modulo `std::fmt`). Questo perché la descrizione dell'errore è ora considerata parte della sua rappresentazione formattata

```
pub trait Error: Debug + Display {  
    fn source(&self) -> Option<&(dyn Error + 'static)> { ... }  
}
```

- Il metodo `source` è pensato per permettere di tracciare la sequenza di fallimenti.

```
use std::error::Error;
use std::fmt;
#[derive(Debug)]
struct CustomError {
    message: String,
}
impl fmt::Display for CustomError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.message)
    }
}
impl Error for CustomError {} // Funzione che restituisce un errore personalizzato
fn do_something() -> Result<(), Box<dyn Error>> {
    // Simuliamo un errore
    let error_message = "Qualcosa è andato storto!";
    Err(Box::new(CustomError {message: error_message.to_string(),}))
}
fn main() {
    match do_something() {
        Ok(_) => println!("Operazione completata con successo."),
        Err(err) => println!("Errore: {}", err),
    }
}
```



Tipi generici (generics)

- Un sistema di tipi stringente facilita la creazione di codice robusto
 - Identificando, in fase di compilazione, quei frammenti di codice che violano il sistema dei tipi
- In certe situazioni, per non violare il sistema dei tipi, occorre replicare una grande quantità di codice generando problemi in fase di manutenzione
 - Occorre, infatti, garantire che eventuali modifiche ad una versione del codice vengano propagate a tutte le altre
- Sia il C++ che Rust permettono di estendere il sistema dei tipi utilizzando una forma di meta-programmazione (detta *template programming* in C++ e *generics* in Rust) grazie alla quale è possibile descrivere strutture dati e funzioni che contengono dati (o che operano su dati) il cui tipo è rappresentato da una meta-variabile (segnaposto), permettendo così di esprimere dei concetti più generali

Funzioni generiche

- In C++, come in Rust, si può definire una funzione in modo che operi su un tipo di dato non ancora precisato

Con questa notazione sintattica dico al compilatore che la funzione fa uso di un tipo generico

```
template <typename T>
T max(
    T t1,
    T t2) {
    return (t1 < t2 ?
        t2 : t1);
}
```

C++

```
fn max<T>(
    t1: T,
    t2: T) -> T where T: Ord {
    return
        if t1 < t2 { t2 }
        else { t1 }
}
```

Rust

Vincolo, ossia condizione necessaria

Tipi generici (generics) in Rust

- In Rust si possono usare costrutti generici per generalizzare i parametri / tipo di ritorno di una funzione o per definire un tipo composto (struct, tupla, enum) basato su parti variabili
 - Si definisce un tipo generico basato sulle meta-variabili T, U, V, ... indicando tali meta-variabili all'interno di parentesi angolari < > dopo il nome del tipo
 - Ciascuna meta-variabile può essere soggetta ad eventuali restrizioni, indicate come l'insieme dei tratti che deve implementare e/o come il tempo di vita e deve garantire
- Se una struttura generica implementa dei metodi, i nomi delle meta-variabili ed i vincoli cui sono soggette vengono ripetuti nel blocco impl

```
struct MyStruct<T> where T: SomeTrait {  
    foo: T,  
    //...  
}
```

```
impl<T> struct MyStruct<T>  
    where T: SomeTrait {  
    fn process(&self) { ... }  
}
```

Tipi generici (in generale)

- Quando il compilatore incontra la definizione di tipi / funzioni generici, **si limita a verificare formalmente la coerenza** del costrutto, senza generare alcun codice
 - Se in qualche parte del programma si utilizza un tipo / funzione generico legando le meta-variabili in esso contenute a tipi concreti, il costrutto generico viene **istanziato**, espandendo la definizione iniziale con i necessari dettagli necessari a generare il codice relativo
 - Se il costrutto generico, in parti diverse del programma, è legato a tipi concreti differenti, il compilatore genera ulteriori espansioni della definizione che, pur avendo una matrice comune, risulteranno indipendenti tra loro
 - Questo processo prende il nome di **monomorfizzazione**
- L'uso di tipi generici facilita il processo di astrazione da parte del programmatore
 - Affidando al compilatore il compito di rifinire i dettagli necessari a gestire specifici tipi di dato

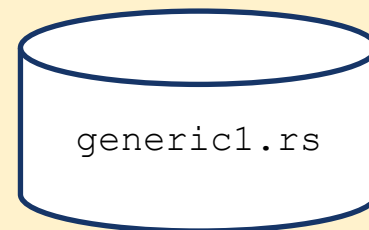
Trait Bound

- Rust offre due modi per specificare i vincoli sui tipi generici
 - Una versione compatta `<T: SomeTrait>`
 - La versione estesa `<T> ... where T: SomeTrait`, posta dopo la lista dei parametri della funzione.
- In entrambi i casi, se è necessario indicare che il tipo deve implementare più tratti, questi possono essere combinati con il segno `+`

```
fn run_query<M: Mapper + Serialize, R: Reducer + Serialize>(
    data: &DataSet, map: M, reduce: R) -> Results
{ ... }
```

```
fn run_query<M,R>( data: &DataSet, map: M, reduce: R) -> Results
    where M: Mapper + Serialize,
           R: Reducer + Serialize
{ ... }
```

```
fn get_value<T>(condition: bool, value: T) -> Option<T> {  
    if condition {  
        Some(value)  
    } else {  
        None  
    }  
}
```

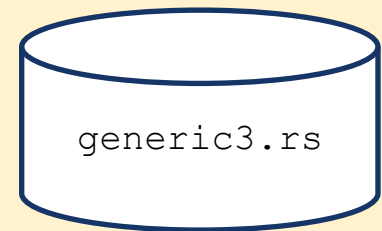


```
fn main() {  
    let condition = true;  
    let value = "Hello, world!";  
    match get_value(condition, value) {  
        Some(v) => println!("Value: {}", v),  
        None => println!("No value found"),  
    }  
  
    match get_value(condition, 42) {  
        Some(v) => println!("Value: {}", v),  
        None => println!("No value found"),  
    }  
}
```

```
fn max<T: PartialOrd>(items: &[T]) -> Option<&T> {
    if items.is_empty() {
        None
    } else {
        let mut max = &items[0];
        for item in items.iter() {
            if item > max { max = item; }
        }
        Some(max)
    }
}

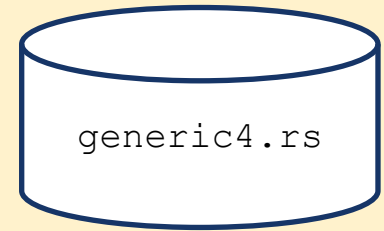
fn my_max <T: std::fmt::Display> (value: Option<&T>)
{ match value {
    Some(max) => println!("Massimo: {}", max),
    None => println!("Sequenza vuota")
}
}

fn main() {
    let numbers = vec![1, 3, 5, 2, 4];
    my_max(max(&numbers));
    let strings = vec!["apple", "banana", "orange", "grape"];
    my_max(max(&strings));
}
```



```
fn sum<T: std::ops::Add<Output = T> + Default + Copy> (items: &[T]) -> T {  
    let mut result= T::default();  
    for item in items {  
        result = result + *item;  
    }  
    result  
}
```

```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
    println!("Somma: {}", sum(&numbers));  
  
    let floats = vec![1.1, 2.2, 3.3, 4.4, 5.5];  
    println!("Somma: {}", sum(&floats));  
}
```



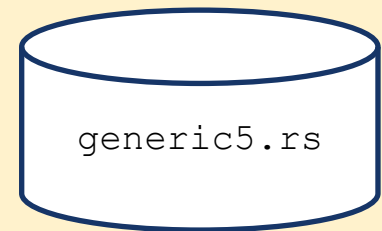
```
#[derive(Debug)]
struct Pair<T, U> {
    first: T,
    second: U,
}

impl<T, U> Pair<T, U> {
    fn new(first: T, second: U) -> Self {
        Pair { first, second }
    }
}

fn main() {
    // Crea una coppia di interi
    let integer_pair = Pair::new(10i32, 20i32);
    println!("Integer pair: {:?}", integer_pair);

    // Crea una coppia di stringhe
    let string_pair = Pair::new("Hello".to_string(), "World".to_string());
    println!("String pair: {:?}", string_pair);

    // Crea una coppia di valori misti
    let mixed_pair = Pair::new(10i32, "Hello".to_string());
    println!("Mixed pair: {:?}", mixed_pair);
}
```



Tratti e tipi generici

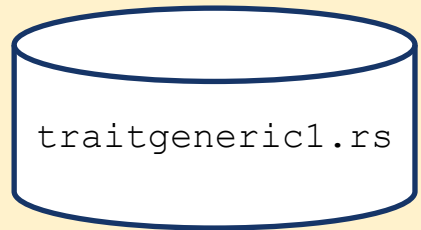
- Sebbene tratti e tipi generici siano due modi per implementare il polimorfismo, tra i due esiste un legame molto più profondo
 - Uno o più tratti possono essere usati come vincoli per limitare l'utilizzo di un tipo generico ai soli casi in cui ha senso farlo
 - Si può definire un **tratto generico**, i cui metodi, cioè, ricevono o restituiscono valori generici, eventualmente vincolati da ulteriori tratti.

```
trait Confrontabile<T> {
    fn confronta(&self, other: &T) -> bool;
}
// Implementazione del tratto Confrontabile per i tipi di dati confrontabili
impl<T> Confrontabile<T> for T
where T: std::cmp::PartialOrd,
{
    fn confronta(&self, other: &T) -> bool {
        self >= other
    }
}

fn main() {
    let x = 5;
    let y = 10;
    println!("x è maggiore o uguale a y? {}", x.confronta(&y));
    let str1 = String::from("Ciao");
    let str2 = String::from("Rust");
    println!("str1 è maggiore o uguale a str2? {}", str1.confronta(&str2));

    let c = 'c';
    let z = 'z';

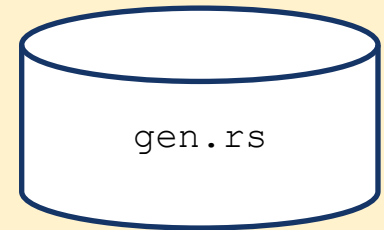
    println!("c è maggiore o uguale a z? {}", c.confronta(&z));
}
```



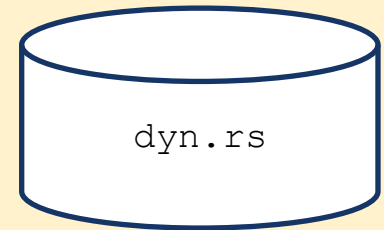
Oggetti-tratto e tipi generici

- Occorre comprendere le somiglianze e le differenze tra una funzione non generica che opera su oggetto-tratto e un'altra generica il cui parametro è vincolato da un tratto e scegliere quando usare una forma o l'altra
 - `fn dynamic_process(w: &mut dyn Write) { ... }`
 - `fn generic_process<T>(w: &mut T) where T:Write { ... }`
- Gli oggetti-tratto richiedono l'uso di un *double pointer* per permetterne l'accesso
 - Ma non richiedono la duplicazione del codice dovuta al processo di monomorfizzazione
- L'uso di strutture dati generiche, in generale, porta a codice più efficiente
 - Non solo perché le chiamate alle funzioni non necessitano di transitare per la VTABLE, ma perché il compilatore, conoscendo il tipo concreto in fase di monomorfizzazione, può generare codice più compatto, valutando il risultato dell'elaborazione delle parti costanti in fase di compilazione e sfruttare tecniche di *code inlining* per ridurre l'impatto dell'invocazione di funzioni
- Non tutti i tratti permettono di definire oggetti-tratto
- Non è possibile definire un oggetto-tratto legato a più tratti disgiunti
 - Mentre è possibile, in una funzione generica, vincolare una meta-variabile ad implementare più tratti


```
trait Suona { fn suona(&self);}  
struct Chitarra;  
struct Pianoforte;  
impl Suona for Chitarra {  
    fn suona(&self) {  
        println!("Chitarra: Strum!");  
    }  
}  
impl Suona for Pianoforte {  
    fn suona(&self) {  
        println!("Pianoforte: Ding!");  
    }  
}  
fn esegui_melodia<T: Suona>(strumento: & T) {  
    strumento.suona();  
}  
fn main() {  
    let chitarra = Chitarra;  
    let pianoforte = Pianoforte;  
    esegui_melodia(&chitarra);  
    esegui_melodia(&pianoforte);  
}
```



```
trait Suona {  
    fn suona(&self);  
}  
struct Chitarra;  
struct Pianoforte;  
impl Suona for Chitarra {  
    fn suona(&self) {  
        println!("Chitarra: Strum!");  
    }  
}  
impl Suona for Pianoforte {  
    fn suona(&self) {  
        println!("Pianoforte: Ding!");  
    }  
}  
fn esegui_melodia(strumento: &dyn Suona) { strumento.suona(); }  
fn main() {  
    let chitarra = Chitarra;  
    let pianoforte = Pianoforte;  
    esegui_melodia(&chitarra);  
    esegui_melodia(&pianoforte);  
}
```





Per saperne di più

- What is generic programming?
 - <https://oswalt.dev/2020/08/what-is-generic-programming/>
- Using generic types in Rust
 - <https://oswalt.dev/2021/06/using-generic-types-in-rust/>
- Polymorphism in Rust
 - <https://oswalt.dev/2021/06/polymorphism-in-rust/>
- Rust Traits: Defining Behavior
 - <https://oswalt.dev/2020/07/rust-traits-defining-behavior/>
- Tour of Rust's standard library traits
 - <https://github.com/pretzelhammer/rust-blog/blob/master/posts/tour-of-rusts-standard-library-traits.md>

Glossario

- **Tratto:** collezione di metodi che definiscono un comportamento che un tipo può implementare
- **Zero-cost abstraction:** linguaggio pensato per ridurre a zero (tranne il caso del dispatch dinamico che si ha nel caso di tratti oggetti) l'overhead introdotto dalle astrazioni di alto livello (come tratti, iteratori, closure e generici)
- **Preludio:** insieme di elementi (tipi, funzioni e tratti) del crate std (libreria standard di Rust) che vengono automaticamente importati in ogni modulo di ogni crate Rust
- **Override:** se un tratto definisce un'implementazione di default di un metodo, è possibile che un tipo utilizzi tale implementazione oppure la ridefinisca
- **Tipo associato ad un tratto:** rappresentano un segnaposto di tipo (diverso dal tipo su cui è implementato il tratto, detto Self)
- **Tipo concreto:** nell'implementazione del tratto per un determinato tipo viene esplicitato il tipo specifico definito come tipo associato al tratto

Glossario

- **Dispatching Statico:** Il compilatore sa esattamente quale funzione chiamare al momento della compilazione
- **Dispatching Dinamico:** la decisione su quale implementazione di un metodo chiamare viene presa a runtime
- **Oggetto Tratto:** riferimento (puntatore doppio) a un valore di un tipo concreto che implementa il tratto, indicato attraverso un tratto (con la notazione `&dyn Tratto` oppure `Box<dyn Tratto>`). Il compilatore specificherà l'oggetto tratto e fornirà il riferimento al dato ed ai metodi che implementano il tratto (attraverso un secondo puntatore alla VTABLE).
- **Astrazione del parametro:** la firma della funzione non riporta il tipo del parametro, ma solo il suo tratto. Il compilatore genera una funzione generica per ogni tipo concreto con cui la funzione viene chiamata.

Glossario

- **Supertratto:** una relazione di supertratto impone che un tratto (supertratto) debba essere implementato per avere l'implementazione del sottotratto. Ad es. Eq è un sottotratto di PartialEq (supertratto). Per avere Eq occorre implementare PartialEq.
- **Sottotratto:** la relazione tra sottotratto e supertratto è definita nella definizione del sottotratto attraverso la notazione : Supertrait. Il sottotratto include tutte le funzionalità del supertratto e ne può aggiungere di proprie
- **Tratto marker:** tratto privo di metodi
- **Tipo wrapper:** un tipo che avvolge (incapsula) un altro tipo
- **Operator Overloading:** meccanismo per ridefinire il comportamento degli operatori standard del linguaggio adottandoli a istanze di tipi personalizzati
- **Deref Coercion:** conversione automatica fatta dal compilatore per trasformare un riferimento ad un tipo T ad un riferimento ad un tipo U, se T implementa il tratto Deref <Target = U>

Glossario

- **Programmazione Generica:** paradigma che permette di scrivere codice che opera su diversi tipi di dati senza specificare il tipo esatto in fase di scrittura
- **Parametro di tipo:** segnaposto per i tipi concreti che verranno specificati quando il codice generico viene utilizzato. I parametri di tipo sono tipicamente rappresentati da nomi di singole lettere maiuscole, come T, U, V, ecc.
- **Tipo generico:** definizione di tipo che contiene dei parametri di tipo
- **Tipo concreto:** tipo utilizzato nell'implementazione al posto del tipo generico
- **Funzione Generica:** funzione che ha come parametro o come valore di ritorno dei parametri di tipo
- **Tratto Generico:** tratto che è parametrizzato su uno o più tipi
- **Trait Bound** (vincolo di tratto): vincolo specificato su un parametri di tipo generico. Esso assicura che il tipo concreto che verrà sostituito al parametro generico implementi uno o più tratti specifici

Glossario

- **Monomorfizzazione:** processo mediante il quale il compilatore prende il codice generico e ne crea copie specializzate per ogni tipo concreto utilizzato con quel codice generico
- **Inline coding:** tecnica adottata dal compilatore per inserire il codice delle funzioni dentro il codice senza inserire la chiamata a funzione.